

Handout on Neural Networks

Dr. Mohammad Mahmudul Alam (MLD)

CSE 445 Machine Learning

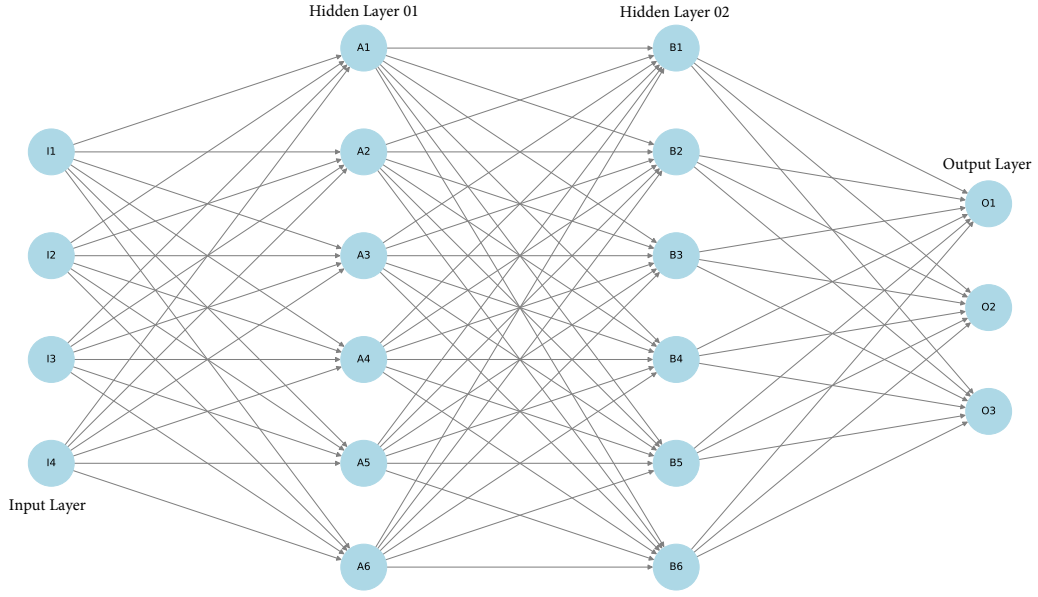


Figure 1: A three-layer neural network with 2 hidden and 1 output layer.

$$W^{(1)} \in \mathbb{R}^{(6 \times 4)} = \begin{bmatrix} 0.32 & -0.45 & 0.67 & -0.89 \\ -0.76 & 0.54 & -0.23 & 0.12 \\ 0.91 & -0.33 & 0.45 & -0.68 \\ -0.11 & 0.72 & -0.55 & 0.39 \\ 0.44 & -0.91 & 0.37 & 0.25 \\ -0.31 & 0.68 & -0.79 & 0.58 \end{bmatrix} \quad b^{(1)} \in \mathbb{R}^{(6 \times 1)} = \begin{bmatrix} 0.15 \\ -0.27 \\ 0.38 \\ -0.42 \\ 0.59 \\ -0.31 \end{bmatrix} \quad (1)$$

$$W^{(2)} \in \mathbb{R}^{(6 \times 6)} = \begin{bmatrix} -0.22 & 0.47 & -0.51 & 0.63 & -0.14 & 0.38 \\ 0.33 & -0.72 & 0.25 & -0.49 & 0.58 & -0.67 \\ -0.44 & 0.29 & -0.87 & 0.12 & -0.51 & 0.73 \\ 0.67 & -0.38 & 0.92 & -0.16 & 0.33 & -0.48 \\ -0.11 & 0.74 & -0.55 & 0.27 & -0.31 & 0.62 \\ 0.49 & -0.82 & 0.43 & -0.28 & 0.59 & -0.64 \end{bmatrix} \quad b^{(2)} \in \mathbb{R}^{(6 \times 1)} = \begin{bmatrix} -0.23 \\ 0.34 \\ -0.51 \\ 0.62 \\ -0.45 \\ 0.29 \end{bmatrix} \quad (2)$$

$$W^{(3)} \in \mathbb{R}^{(3 \times 6)} = \begin{bmatrix} 0.12 & -0.35 & 0.48 & -0.67 & 0.23 & -0.54 \\ -0.42 & 0.61 & -0.19 & 0.85 & -0.39 & 0.27 \\ 0.53 & -0.71 & 0.32 & -0.48 & 0.76 & -0.61 \end{bmatrix} \quad b^{(3)} \in \mathbb{R}^{(3 \times 1)} = \begin{bmatrix} 0.27 \\ -0.36 \\ 0.45 \end{bmatrix} \quad (3)$$

1 Inference

Figure 1 shows a three-layer neural network with 2 hidden and 1 output layer. The network is trained using the Iris dataset. As we know, the Iris dataset has 4 features and 3 iris species classes (Setosa, Versicolor, and Virginica). That's why the network has 4 input nodes and 3 output nodes. Both of the hidden layers have 6 nodes, i.e., hidden units. The weight W and bias b of each layer are provided. Consider the following instance $\mathbf{X}$ from the Iris dataset.

$$\mathbf{X} \in \mathbb{R}^{(4 \times 1)} = \begin{bmatrix} 4.7 \\ 3.2 \\ 1.6 \\ 0.2 \end{bmatrix} \quad (4)$$

Let's perform the inference to figure out the class label of the instance. We start by calculating the output of the first layer, $Z^{(1)}$.

$$Z^{(1)} = W^{(1)}\mathbf{X} + b^{(1)} = \begin{bmatrix} 1.11 \\ -2.46 \\ 4.18 \\ 0.56 \\ 0.39 \\ -0.74 \end{bmatrix} \quad (5)$$

Since this is a hidden layer, let's apply ReLU activation.

$$A^{(1)} = \text{ReLU}(Z^{(1)}) = \begin{bmatrix} 1.11 \\ 0.0 \\ 4.18 \\ 0.56 \\ 0.39 \\ 0.0 \end{bmatrix} \quad (6)$$

We will repeat the calculations for the second hidden layer where our inputs will be $A^{(1)}$ and outputs $Z^{(2)}$.

$$Z^{(2)} = W^{(2)}A^{(1)} + b^{(2)} = \begin{bmatrix} -2.31 \\ 1.7 \\ -4.77 \\ 5.25 \\ -2.84 \\ 2.7 \end{bmatrix} \quad (7)$$

Again, apply the ReLU activation.

$$A^{(2)} = \text{ReLU}(Z^{(2)}) = \begin{bmatrix} 0.0 \\ 1.7 \\ 0.0 \\ 5.25 \\ 0.0 \\ 2.7 \end{bmatrix} \quad (8)$$

Finally, for the output layer, we will calculate $Z^{(3)}$.

$$Z^{(3)} = W^{(3)}A^{(2)} + b^{(3)} = \begin{bmatrix} -5.3 \\ 5.87 \\ -4.92 \end{bmatrix} \quad (9)$$

Since, this is the final layer, and it is a multi-class classification problem, the appropriate activation function is Softmax. Let's apply softmax on $Z^{(3)}$.

$$\hat{Y} = A^{(3)} = \text{softmax}(Z^{(3)}) = \begin{bmatrix} 0.00 \\ 1.00 \\ 0.00 \end{bmatrix} \quad (10)$$

This is the final output $\hat{Y}$ provides a probability distribution of each class. The max probability (100.00%) belongs to the second class which is "Versicolor". Therefore, the input instance $\mathbf{X}$ belongs to the Versicolor species.

2 Training

Let's see the steps to train this neural network. The first step of training any deep learning network is forward pass, i.e., forward propagation which is the same as our inference procedure. Let's assume all the weights and biases are initialized randomly. With those initial random weights and biases, we will calculate $\hat{Y}$ for training instances. Using the ground truth labels of each instance, we will formulate our cost function where Y is the ground truth label in one-hot encoding format and M is the total no. of instances used in forward pass, i.e., batch size.

$$Cost, J = - \sum_{i=1}^M Y \log \hat{Y} \quad (11)$$

Next, the gradient of the cost function with respect to the parameters is calculated through backpropagation. Modern deep learning tools, e.g., PyTorch/TensorFlow use Automatic Differentiation (Autodiff) algorithm to calculate the gradients. The steps of calculation if lengthy and cumbersome for us, however, computers can perform them quite fast.

OK. Let's say we have calculated all the gradients:

$$\frac{\partial J}{\partial W^{(1)}}, \frac{\partial J}{\partial W^{(2)}}, \frac{\partial J}{\partial b^{(1)}}, \frac{\partial J}{\partial b^{(2)}} \quad (12)$$

After calculating the gradients, updating parameters is quite straightforward.

$$W^{(l)} = W^{(l)} - lr * \frac{\partial J}{\partial W^{(l)}} \quad (13)$$

$$b^{(l)} = b^{(l)} - lr * \frac{\partial J}{\partial b^{(l)}} \quad (14)$$

Here, l is the layer number and lr is the learning rate.

Cool. Now we have to repeat these processes until the model converges. In summary, here are the steps of training any neural network.

1. Forward propagation
2. Estimate loss
3. Compute gradients of the loss w.r.t. parameters
4. Update parameters (weights and biases)